Replicação do *Reflective Gaussian Splatting* para Objetos Reflexivos: Primeiros Resultados e Propostas

João Pedro Maciel Pimenta
Centro de Informática
Universidade Federal de Pernambuco
Recife, Brasil
jpmp3@cin.ufpe.br

Francisco Paulo Magalhães Simões
Centro de Informática
Universidade Federal de Pernambuco
Recife, Brasil
fpms@cin.ufpe.br

***Resumo*—Métodos de Gaussian Splatting reconstroem cenas 3D a partir de fotos e as renderizam em tempo real, mas têm dificuldade com objetos espelhados, como metais polidos. O Reflective Gaussian Splatting (Ref-Gaussian) trata esse problema modelando explicitamente o material e a luz refletida. Este relatório descreve o método, aponta uma limitação na forma como ele decide se um reflexo está bloqueado por outra parte do objeto e propõe duas formas de melhorá-la. Como primeiro passo, o código oficial foi executado em um notebook comum e os resultados do artigo foram replicados nas seis cenas do conjunto Shiny Blender: o SSIM médio é idêntico ao publicado, o LPIPS médio difere em 0,001 e o PSNR médio fica cerca de 0,6 dB abaixo, diferença que se explica em boa parte pelo protocolo de avaliação. Mesmo assim, a replicação fica 4,1 dB acima do 3DGS original na média reportada pelo artigo. A análise dos erros mostra que eles se concentram nos pontos em que o método usa a luz refletida pelo próprio objeto, o que só acontece em objetos espelhados com partes que se refletem, e que os materiais estimados saem ruidosos mesmo quando a imagem final é fiel.**

***Index Terms*—Gaussian Splatting, superfícies reflexivas, reconstrução 3D, renderização.**

I. Introdução

O 3D Gaussian Splatting (3DGS) [1] representa uma cena como um grande conjunto de "manchas" 3D em formato de elipsoide, chamadas Gaussianas, cada uma com posição, tamanho, cor e transparência. A partir de várias fotos de um objeto, o método ajusta essas Gaussianas até que, projetadas na tela, reproduzam as fotos. O resultado pode ser visto de novos ângulos em tempo real.

O 3DGS funciona mal com objetos espelhados. A cor de cada Gaussiana muda com o ângulo de visão de forma simplificada, o que não é suficiente para representar reflexos nítidos. O método acaba "pintando" o reflexo como se fosse geometria atrás da superfície.

O Ref-Gaussian [3] resolve boa parte desse problema e é hoje um dos melhores métodos para objetos reflexivos. Por isso ele foi escolhido como *baseline*, isto é, o método de referência que a dissertação pretende melhorar. Este relatório tem dois objetivos: explicar o método e as propostas de melhoria (Seção II) e reproduzir os resultados do artigo antes de qualquer modificação (Seções III e IV).

II. Solução Proposta

A. Como o Ref-Gaussian funciona

A Fig. 1 resume o método. Ele tem quatro ideias principais.

1) Gaussianas achatadas. Em vez de elipsoides, o método usa discos planos (Gaussianas 2D [2]). Um disco tem uma direção perpendicular bem definida, a *normal*, que diz para onde a superfície "aponta". Saber a normal é indispensável para calcular reflexos.

2) Material em vez de cor. Cada disco guarda propriedades físicas de material: a cor base (*albedo*), o quanto o material é metálico (*metalicidade*) e o quanto ele é áspero (*rugosidade*). Um metal polido tem metalicidade alta e rugosidade baixa e reflete como um espelho; um metal escovado tem rugosidade maior e reflete de forma borrada.

3) Cálculo da luz por pixel. Primeiro, as propriedades de todos os discos são projetadas na tela, formando um mapa de material por pixel. Só depois a cor é calculada, uma vez por pixel, usando um modelo físico de reflexão e um mapa da luz que vem do ambiente ao redor do objeto. Isso produz reflexos mais nítidos do que calcular a cor disco a disco.

4) Reflexos entre partes do objeto. Um ponto da superfície pode refletir o ambiente distante ou outra parte do próprio objeto, como a lateral de uma torradeira refletindo a fatia de pão. Para decidir qual dos dois casos acontece, o método lança *um* raio na direção do reflexo e verifica se ele bate no objeto. O resultado é a *visibilidade* V, que vale 1 se o raio escapa para o ambiente e 0 se é bloqueado:

$$L_{\text{reflexo}} = V \cdot L_{\text{ambiente}} + (1 - V) \cdot L_{\text{objeto}}. \tag{1}$$

Para lançar esse raio, o método precisa de uma malha de triângulos da superfície, que é recalculada a partir das Gaussianas a cada 2 mil iterações de treino. Um detalhe importante, conferido no código: a luz L_{objeto} não é buscada no ponto que o raio atinge. Cada disco aprende sua própria "luz indireta" como uma função da direção de reflexão (harmônicos esféricos de grau 3), ajustada apenas pelas fotos de treino.

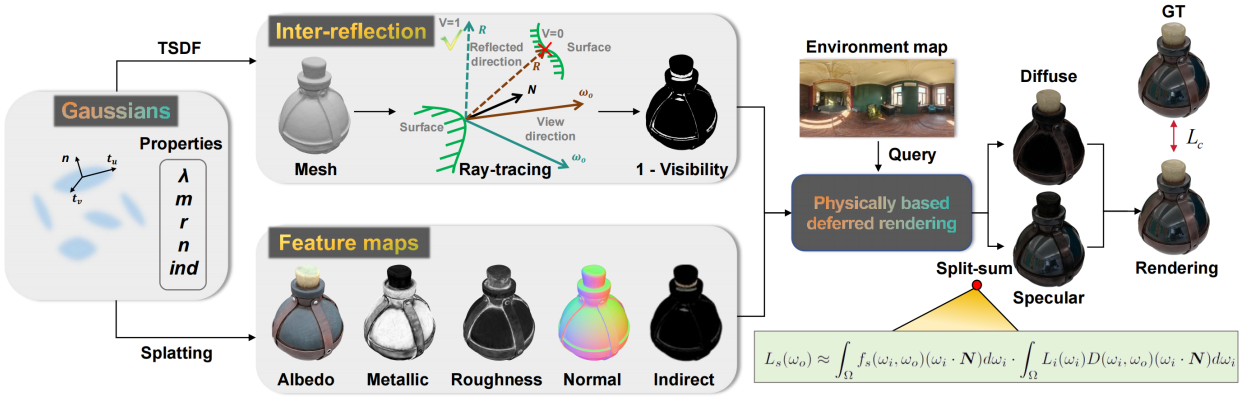


Figura 1. Visão geral do Ref-Gaussian. As Gaussianas guardam propriedades de material, que são projetadas em mapas por pixel; a cor final é calculada a partir desses mapas, da luz do ambiente e dos reflexos entre partes do objeto. Fonte: Yao et al. [3].

B. Limitação e propostas

O próprio artigo aponta uma limitação na Eq. (1). Um reflexo real não vem de uma única direção, mas de um pequeno "cone" de direções, que é mais largo quanto mais áspero é o material. Testar só o raio central é uma aproximação, e os autores observam que ela introduz ruído nas propriedades de material estimadas. Além disso, recalcular a malha periodicamente tem custo e cria uma segunda representação da geometria, separada das Gaussianas.

A partir disso, a dissertação propõe investigar:

P1 – Visibilidade em todo o cone de reflexão. Lançar alguns raios distribuídos dentro do cone, em vez de um só, e usar a média, $\bar{V} = \frac{1}{K}\sum_{k=1}^{K} V_k$. O número de raios K cresceria com a rugosidade; para um espelho perfeito, $K = 1$ recupera o método original.

P2 – Visibilidade e luz indireta direto nas Gaussianas. Calcular se o raio é bloqueado diretamente a partir das Gaussianas, sem gerar a malha, aproveitando trabalhos recentes de traçado de raios sobre Gaussianas [8]. Com o ponto atingido em mãos, a luz indireta pode ser buscada nas Gaussianas desse ponto, em vez de ser um valor aprendido à parte. Os resultados da Seção IV mostram que é justamente essa luz aprendida que mais erra.

Uma terceira direção, a extensão para objetos reflexivos em movimento, fica como trabalho futuro.

III. Experimentos

O objetivo desta etapa é *replicar* o baseline, ou seja, verificar se os resultados do artigo se repetem quando o código é executado por outra pessoa, em outro computador.

Ambiente. Foi usado o código oficial do Ref-Gaussian, sem modificações no método, em um notebook com GPU NVIDIA RTX 4050 (6 GB), Windows 11 e CUDA 11.8. O código foi feito para Linux, e foi necessário ajustar opções de compilação das suas partes em CUDA para que funcionassem no Windows. Esses ajustes estão registrados em scripts no repositório do projeto.

Dados. Usou-se o conjunto Shiny Blender [6], com seis objetos sintéticos muito reflexivos. Cada objeto tem 100 imagens para treino e 200 para teste, todas com 800×800 pixels. As seis cenas foram treinadas: *ball* (esfera), *car* (carro de brinquedo), *coffee* (xícara), *helmet* (capacete), *teapot* (bule) e *toaster* (torradeira).

Treino. Cada cena foi treinada por 50 mil iterações, com as configurações padrão do código oficial. As primeiras 18 mil iterações usam um modo mais simples, que serve para estabilizar a forma do objeto; os reflexos entre partes do objeto são ativados a partir da iteração 20 mil.

Métricas. Cada imagem de teste gerada pelo modelo é comparada com a imagem real usando três métricas comuns na área: PSNR, que mede o erro pixel a pixel (maior é melhor); SSIM, que mede a semelhança de estrutura (maior é melhor); e LPIPS, que mede a diferença percebida por uma rede neural treinada para imitar a percepção humana (menor é melhor).

IV. Resultados e Discussão

Os resultados do artigo se repetem. A Tabela I compara os valores obtidos com os publicados nas seis cenas. Na média, o SSIM (0,973) é idêntico ao do artigo, o LPIPS difere em 0,001 (0,057 contra 0,056) e o PSNR fica 0,57 dB abaixo (34,46 contra 35,04 dB, Tabela I). Em *car* e *teapot* os resultados ficam iguais ou acima dos publicados; em *coffee* e *toaster* ficam a menos de 0,6 dB; quase todo o déficit vem de *helmet* ($-1{,}87$ dB) e *ball* ($-0{,}94$ dB).

A vantagem sobre os outros métodos se mantém. A Tabela III coloca a replicação ao lado dos métodos comparados no artigo. Mesmo com o déficit de 0,57 dB, ela fica 4,1 dB acima do 3DGS original na média (34,46 contra 30,36 dB), com as maiores diferenças justamente nos objetos mais espelhados: +8,4 dB na *ball* e +6,5 dB na *toaster*. Em quatro das seis cenas (*ball*, *car*, *teapot* e *toaster*) o resultado fica acima do melhor método anterior ao Ref-Gaussian segundo o próprio artigo (3DGS-DR [5]); em *coffee* e *helmet* fica abaixo dele. Na *teapot*, quase convexa e pouco espelhada, todos os métodos ficam entre 45 e 47 dB, e a escolha do método quase não importa.

O treino tem duas fases visíveis. A Fig. 4 mostra o PSNR de teste durante o treino. Até 18 mil iterações o método usa o

Tabela I

REPLICAÇÃO DO REF-GAUSSIAN NO SHINY BLENDER: VALORES DO ARTIGO [3] E OBTIDOS NESTE TRABALHO. Δ É A DIFERENÇA DE PSNR EM RELAÇÃO AO ARTIGO. MODELO FINAL (50 MIL ITERAÇÕES), AVALIADO COM O `EVAL.PY` OFICIAL.

Métrica	Origem	Ball	Car	Coffee	Helmet	Teapot	Toaster	Média
PSNR ↑	Artigo	37,01	31,04	34,63	32,32	47,16	28,05	35,04
	Replicação	36,07	31,25	34,29	30,45	47,19	27,51	34,46
	Δ	-0,94	+0,21	-0,34	-1,87	+0,03	-0,54	-0,57
SSIM ↑	Artigo	0,981	0,964	0,976	0,971	0,998	0,948	0,973
	Replicação	0,985	0,966	0,976	0,965	0,998	0,948	0,973
LPIPS ↓	Artigo	0,098	0,033	0,076	0,049	0,006	0,074	0,056
	Replicação	0,089	0,031	0,077	0,061	0,006	0,074	0,057

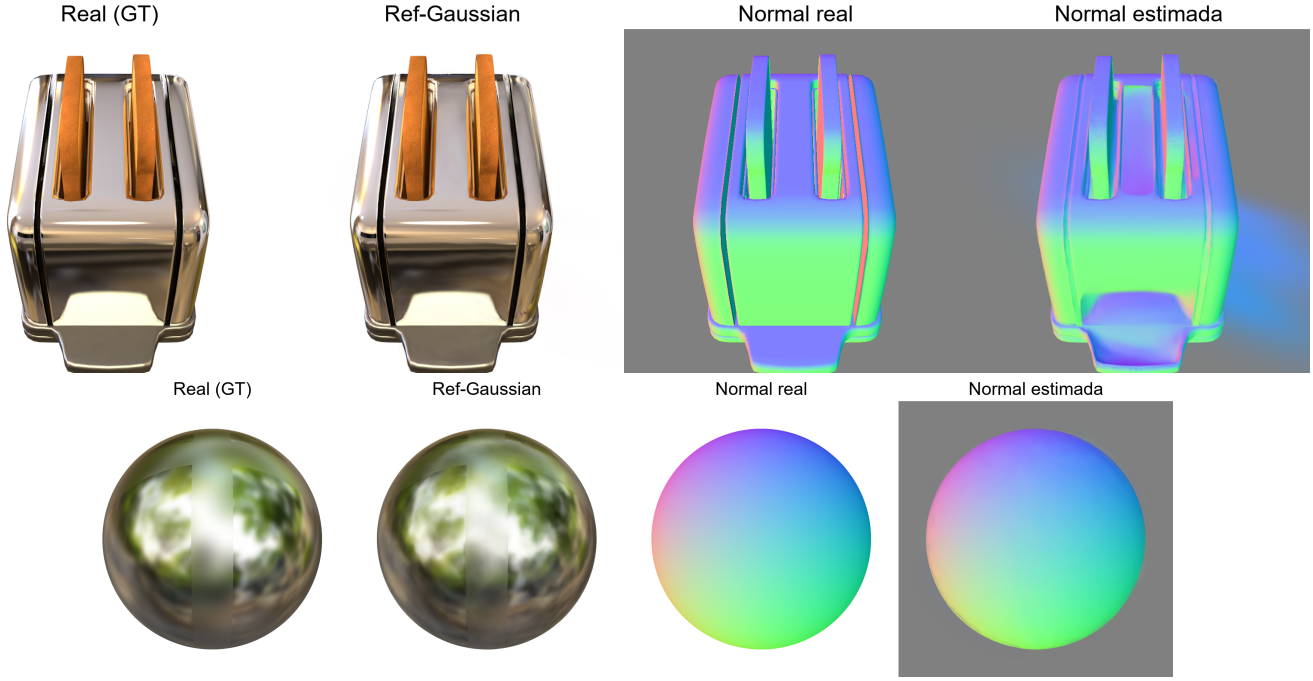


Figura 2. Resultados nas cenas *toaster* (acima) e *ball* (abaixo), na primeira imagem de teste. Da esquerda para a direita: imagem real, imagem gerada pelo Ref-Gaussian, normal real e normal estimada (as cores codificam a direção da superfície).

Tabela II

EFEITO DO CHECKPOINT E DA MALHA USADA NA VISIBILIDADE, AVALIADO NAS MESMAS 200 VISTAS DE TESTE. O `EVAL.PY` OFICIAL USA AS GAUSSIANAS E A MALHA DA ÚLTIMA ITERAÇÃO (50K).

Cena	Gaussianas / malha	PSNR	SSIM	LPIPS
Toaster	40k / 40k	**28,08**	**0,951**	**0,068**
	50k / 48k	27,94	0,951	0,071
	50k / 50k (`eval.py`)	27,51	0,948	0,074
	Artigo	28,05	0,948	0,074
Helmet	40k / 40k	**32,09**	**0,971**	**0,050**
	50k / 48k	31,24	0,968	0,054
	50k / 50k (`eval.py`)	30,45	0,965	0,061
	Artigo	32,32	0,971	0,049

Tabela III

PSNR (DB) NO SHINY BLENDER: OUTROS MÉTODOS, SEGUNDO A TABELA 1 DO ARTIGO [3], E A REPLICAÇÃO DESTE TRABALHO. EM NEGRITO, O MAIOR VALOR DE CADA COLUNA.

Método	Ball	Car	Coffee	Helmet	Teapot	Toaster	Média
3DGS [1]	27,65	27,26	32,30	28,22	45,71	20,99	30,36
2DGS [2]	25,97	26,38	32,31	27,42	44,97	20,42	29,58
Ref-NeRF [6]	33,16	30,44	33,99	29,94	45,12	26,12	33,13
GShader [4]	30,99	27,96	32,39	28,32	45,86	26,28	31,97
3DGS-DR [5]	33,43	30,48	34,53	31,44	47,04	26,76	33,95
Ref-Gaussian (artigo)	**37,01**	31,04	**34,63**	**32,32**	47,16	**28,05**	**35,04**
Ref-Gaussian (replicação)	36,07	**31,25**	34,29	30,45	**47,19**	27,51	34,46

modo mais simples; na troca para o cálculo de luz por pixel o PSNR cai por algumas iterações (de 24,3 para 20,2 dB na *toaster*) e logo passa do patamar anterior. A partir daí as curvas sobem devagar até o fim, exceto em *toaster* e *helmet*, em que atingem um pico perto de 43 mil iterações e depois caem, como discutido a seguir.

Parte do desvio vem do protocolo, não do método. Durante o treino, o PSNR de teste chega ao valor do artigo e depois cai: na *helmet* ele atinge 32,29 dB por volta da iteração

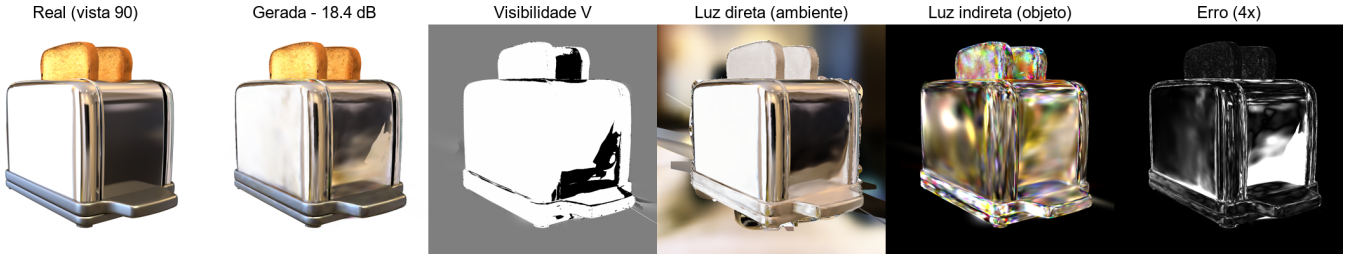


Figura 3. Pior imagem de teste da *toaster* (18,4 dB). Da esquerda para a direita: real, gerada, visibilidade V (preto: raio bloqueado), luz direta do ambiente, luz indireta aprendida e erro. A luz do ambiente já tem a face direita escura, como na foto; o erro aparece na mancha em que $V = 0$ e o método usa a luz indireta, mais clara.

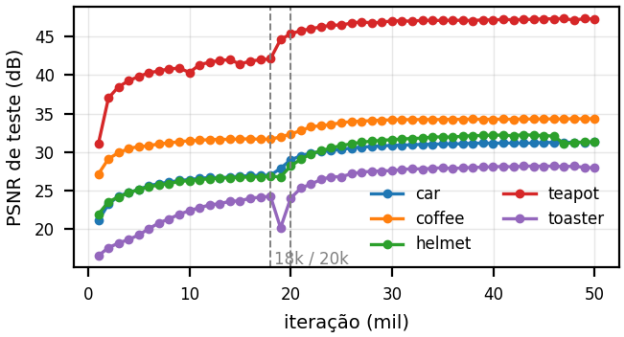


Figura 4. PSNR de teste ao longo do treino, nas cinco cenas em que foi registrado. A queda entre 18 mil e 20 mil iterações (linhas tracejadas) é a troca para o cálculo de luz por pixel com reflexos entre partes do objeto.

43 mil e termina em 31,36 dB. Avaliando os checkpoints salvos (Tabela II), o modelo com 40 mil iterações supera o artigo nas três métricas na *toaster* e fica a 0,23 dB dele na *helmet*, contra 1,87 dB com o modelo final. Há ainda um segundo efeito: o `eval.py` usa a malha extraída na última iteração, e só trocar a malha da iteração 48 mil por essa custa 0,4 a 0,8 dB. Os números do artigo são, portanto, praticamente alcançáveis com o código publicado; boa parte da diferença está em que ponto do treino e com qual malha se mede.

Dá para treinar em um notebook. Cada cena levou entre 49 minutos (*teapot*) e 1h46min (*toaster*). O artigo relata cerca de 35 minutos em uma GPU profissional (NVIDIA A6000). Os experimentos da dissertação são viáveis com o equipamento disponível. O mesmo código também foi executado em um contêiner Docker com Linux e CUDA 11.8, na mesma máquina, sem os ajustes de compilação do Windows. Com um cronograma reduzido para 10 mil iterações, em que todas as fases do treino são encurtadas na mesma proporção, a *teapot* chega a 45,04 dB (SSIM 0,997, LPIPS 0,009) em 13 minutos, contra 47,19 dB em 49 minutos com o cronograma completo. Isso permite testar mudanças no método rapidamente antes de um treino completo.

A média esconde erros localizados. Na *toaster*, o PSNR varia de 18,4 a 33,5 dB entre as 200 imagens de teste, e 32 delas ficam abaixo de 25 dB; na *helmet*, são 24 abaixo de 25 dB, com o erro concentrado na viseira espelhada. Nas cenas *ball*, *coffee* e *teapot* nenhuma imagem fica abaixo de 29 dB.

As quedas fortes só aparecem em objetos espelhados que se refletem. A esfera (*ball*) é convexa: nenhum ponto da superfície enxerga outra parte do objeto, então a visibilidade vale sempre $V = 1$, e nenhuma imagem de teste fica abaixo de 31,6 dB. O bule (*teapot*), quase convexo, e a xícara (*coffee*), de cerâmica e menos espelhada, também não têm quedas. Elas aparecem na torradeira e no capacete, objetos espelhados com partes que se refletem umas nas outras, ou seja, exatamente onde a Eq. (1) usa o termo L_{objeto}.

Onde o método erra: a luz refletida pelo próprio objeto. Para testar a hipótese da visibilidade, o mapa V foi exportado nas 200 vistas da *toaster*. Nos pixels em que o raio é dado como bloqueado ($V = 0$), o erro médio é 2,2 vezes o dos pixels visíveis, e 4,3 vezes na pior imagem (Fig. 3). Nessa imagem, a luz do ambiente já tinha o tom escuro correto da face direita; o erro surge numa mancha em que $V = 0$ e o método passa a usar a luz indireta, que sai clara. A hipótese inicial, de um raio que "escapa" quando deveria ser bloqueado, estava errada: o problema está no que acontece *depois* do bloqueio. Um teste adicional descartou que essas oclusões fossem um erro numérico (o raio batendo na própria superfície de onde parte): deslocar a origem do raio em até 1% do tamanho da cena praticamente não mudou nada (PSNR médio de 27,51 para 27,54 dB). As oclusões são reais na malha; o que erra é a luz indireta, que é um valor aprendido por disco e não a luz do ponto atingido.

A imagem fica fiel, mas os materiais saem ruidosos. A Fig. 5 mostra os mapas intermediários da *toaster*. O método separa bem o que é metal (metalicidade alta e rugosidade baixa) do que é pão, e o erro da imagem final é pequeno. Os materiais, porém, têm defeitos que não aparecem na imagem: cores falsas no albedo do metal, manchas de rugosidade sobre uma superfície uniforme e um halo de Gaussianas fora do objeto. Isso concorda com a limitação apontada pelos autores (ruído nas propriedades de material) e importa para aplicações que reutilizam o material, como reiluminar o objeto ou inseri-lo em uma cena de realidade aumentada.

O ambiente é recuperado só pelos reflexos. O mapa de luz do ambiente aprendido na *toaster* (Fig. 6) mostra uma sala reconhecível, com janelas, luminárias, uma cama e uma poltrona, embora borrada nas regiões que quase não aparecem refletidas no objeto. É um indício de que a separação entre

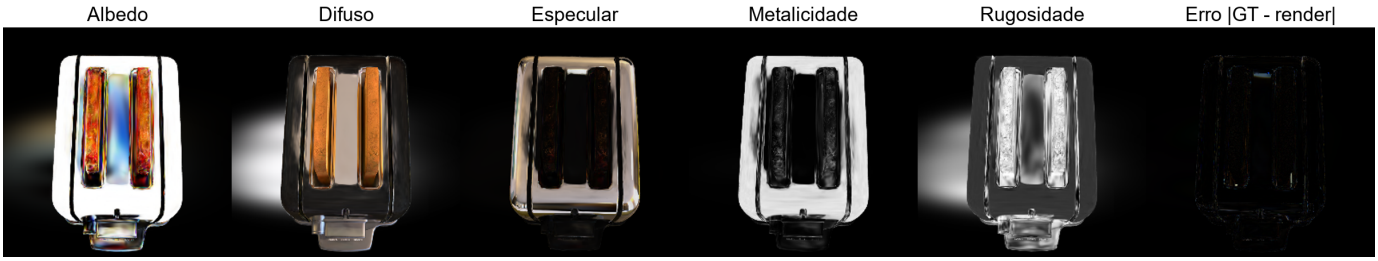


Figura 5. Decomposição estimada pelo Ref-Gaussian na *toaster*: albedo, parcela difusa, parcela especular, metalicidade, rugosidade e erro da imagem final. O metal e o pão são bem separados, mas o albedo tem cores falsas no metal e a rugosidade tem manchas sobre uma superfície que deveria ser uniforme; o halo claro à esquerda são Gaussianas fora do objeto.



Figura 6. Mapa de ambiente aprendido na *toaster* (projeção equirretangular), reconstruído apenas a partir dos reflexos no objeto.

material e luz funciona nas regiões bem observadas.

O que isso muda nas propostas. A transição brusca entre $V = 0$ e $V = 1$ é o que P1 suaviza, e a luz indireta aprendida é o que P2 pode substituir, buscando a luz no ponto em que o raio bate. Esses resultados reforçam P2 como a linha mais promissora.

Próximos passos. (i) Repetir a análise de visibilidade na *helmet* e medir quanto do erro está nos pixels com $V = 0$ em todas as cenas; (ii) replicar nos conjuntos Glossy Synthetic [7] e Shiny Blender Real, usando também os checkpoints intermediários; (iii) estudar a base teórica (3DGS, Gaussianas 2D e PBR); e (iv) implementar um protótipo de P2 que busque a luz indireta no ponto atingido e compará-lo com o baseline.

V. Conclusão

O Ref-Gaussian foi escolhido como ponto de partida da dissertação. O código oficial foi colocado para funcionar em um notebook com Windows e os resultados do artigo foram reproduzidos nas seis cenas do Shiny Blender, com SSIM médio idêntico e LPIPS médio a 0,001 dos publicados; com o checkpoint de 40 mil iterações, as duas cenas mais difíceis chegam ao nível do artigo, e a vantagem sobre o 3DGS original (4,1 dB na média) se confirma. A análise dos erros mostra que eles se concentram onde o método usa a luz refletida pelo próprio objeto, o que só acontece em objetos espelhados com partes que se refletem, e que a causa não é o teste de bloqueio em si, mas a luz indireta aprendida por disco. Os materiais estimados ainda saem ruidosos mesmo quando a imagem é fiel, o que limita aplicações como reiluminação. As propostas da dissertação, suavizar a visibilidade (P1) e calcular visibilidade e luz indireta direto nas Gaussianas (P2), atacam exatamente esse ponto.

Referências

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, 2023.

[2] B. Huang, Z. Yu, A. Chen, A. Geiger, and S. Gao, "2D Gaussian splatting for geometrically accurate radiance fields," in *Proc. ACM SIGGRAPH*, 2024.

[3] Y. Yao, Z. Zeng, C. Gu, X. Zhu, and L. Zhang, "Reflective Gaussian splatting," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025.

[4] Y. Jiang, J. Tu, Y. Liu, X. Gao, X. Long, W. Wang, and Y. Ma, "GaussianShader: 3D Gaussian splatting with shading functions for reflective surfaces," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2024.

[5] K. Ye, Q. Hou, and K. Zhou, "3D Gaussian splatting with deferred reflection," in *Proc. ACM SIGGRAPH*, 2024.

[6] D. Verbin, P. Hedman, B. Mildenhall, T. Zickler, J. T. Barron, and P. P. Srinivasan, "Ref-NeRF: Structured view-dependent appearance for neural radiance fields," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022.

[7] Y. Liu, P. Wang, C. Lin, X. Long, J. Wang, L. Liu, T. Komura, and W. Wang, "NeRO: Neural geometry and BRDF reconstruction of reflective objects from multiview images," *ACM Trans. Graph.*, vol. 42, no. 4, 2023.

[8] N. Moenne-Loccoz, A. Mirzaei, O. Perel, R. de Lutio, J. M. Esturo, G. State, S. Fidler, N. Sharp, and Z. Gojcic, "3D Gaussian ray tracing: Fast tracing of particle scenes," *ACM Trans. Graph.*, vol. 43, no. 6, 2024.